

Week 4 | GUI & Problem-Solving Agent

***CLO - 2:** Apply classical AI techniques, including uninformed and informed search algorithms, minimax, and neural networks, to solve structured problems.*

Submission Guidelines:

1. Attach the link of the deployed streamlit application.
2. **Download the .ipynb file** from Jupyter Notebook.
3. Submit the **.ipynb**, and **application Link** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**

Task 1: Streamlit-Based Exploratory Data Analysis Interface

Develop & Deploy an interactive GUI system for exploratory data analysis (EDA) using Streamlit. The system should facilitate dataset ingestion, metadata inspection, and primary visual pattern recognition, providing end-users with a structured interface for initial data exploration.

Implementation Requirements:

1. Dataset Ingestion & Metadata Inspection

- **File Upload Interface:** Implement a secure file upload module supporting CSV format. Validate that the uploaded file is correctly formatted.
- **Dataset Overview Panel:** Display key metadata including:
 - Dimensional properties (number of rows and columns)
 - Column-wise data types
 - Count of missing values per attribute
 - Basic statistical summary for numerical attributes (mean, median, min, max)

***Note:** Test your implementation using the titanic.csv dataset.*

2. Attribute Selection & Classification

- **Dynamic Feature Selector:** Provide a dropdown to select a single column for analysis.
- **Automated Attribute Typing:** Detect whether the selected column is numerical or categorical, and update the visualization module accordingly.

3. Visualization Module

- **Conditional Rendering:**

- **Numerical Attributes:** Generate a **histogram** depicting the value distribution. Include axis labels and a descriptive title.
- **Categorical Attributes:** Generate a **bar chart** representing frequency counts of unique values. Include axis labels and percentage display (optional).
- Ensure that visualizations are updated dynamically upon attribute selection.

4. Interface Layout & Organization

- **Sidebar Panel:** Should have all interactive controls, including file upload and column selection.
 - **Top section:** Dataset preview (first 5 rows) and metadata display
 - **Bottom section:** Conditional visualization canvas
- **Main Content Area:**
 - **Top section:** Dataset preview (first 5 rows) and metadata display
 - **Bottom section:** Conditional visualization canvas

Deliverables

1. Fully functional deployed Streamlit EDA Application meeting the above specifications.
2. Screenshots of the interface with at least one numerical and one categorical attribute plotted.

Expected Output:

Exploratory Data Analysis Interface

Dataset Preview & Metadata

First 5 Rows:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	None	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925	None	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05	None	S

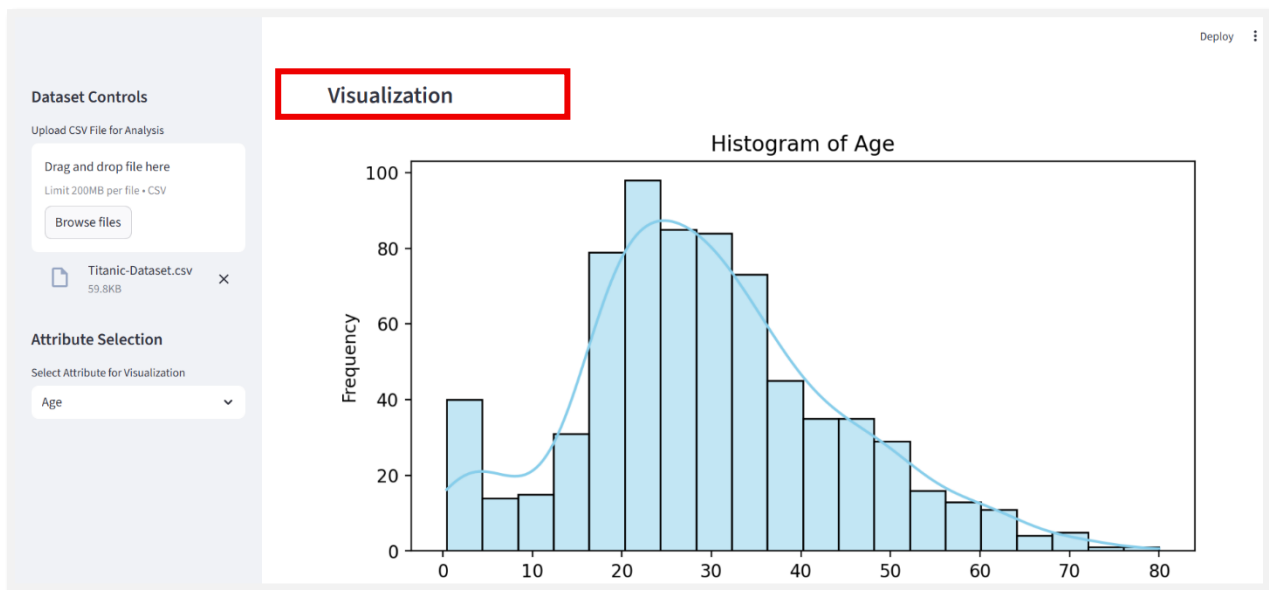
Shape: (891, 12)

Column Data Types:

	Data Type
PassengerId	int64
Survived	int64
Pclass	int64

Missing Values per Column:

	Missing Count	Missing %
Pclass	0	0
Name	0	0
Sex	0	0
Age	177	19.8653
SibSp	0	0
Parch	0	0
Ticket	0	0
Fare	0	0



Task 2: Cloud Resource Deployment Planning Agent

Objective

Design and implement a **Problem-Solving Agent** that plans the deployment of an AI application onto cloud infrastructure.

Scenario

A cloud service provider receives deployment requests for machine learning applications. Before deployment, several sequential operations must be completed, including resource allocation, virtual machine provisioning, dependency installation, model deployment, and service verification. The planning agent must determine a valid execution sequence that results in a successfully deployed AI service.

Specifications

Goal Formulation

Goal State:

AI application deployed successfully.

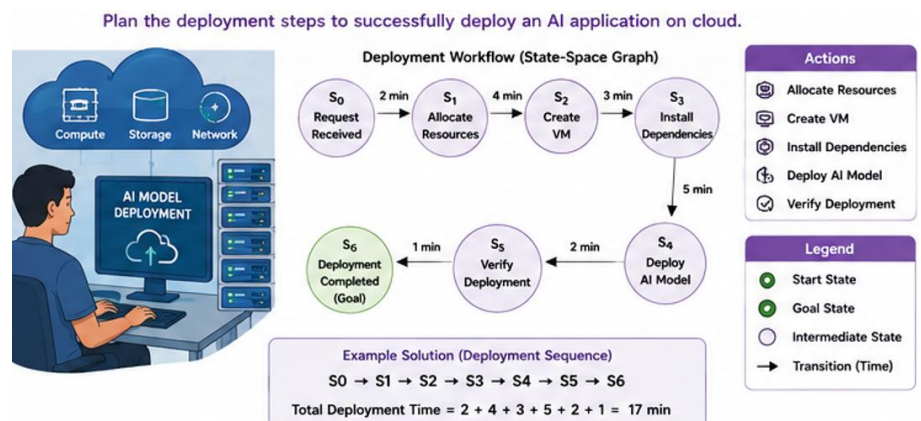
Performance Measure:

- Minimize deployment time.
- Reduce deployment cost.

Problem Formulation

Initial State:

Deployment request received.



Possible Actions:

- Allocate Resources
- Create Virtual Machine
- Install Dependencies
- Deploy AI Model
- Verify Deployment

Transition Model:

Each completed action updates the deployment state.

Goal Test:

Deployment status equals **Completed**.

Path Cost:

Each deployment operation has an associated execution time.

Implementation Requirements

Students shall:

- Represent deployment states programmatically.
- Generate the complete deployment state space.
- Display the execution sequence.
- Calculate total deployment cost.

Task 3: University Course Registration Planning Agent

Objective

Develop a **Problem-Solving Agent** that assists students in completing the university course registration process.

Scenario

An intelligent university registration assistant guides students through a structured registration workflow. Before enrollment is finalized, several mandatory academic processes must be completed, including authentication, prerequisite verification, course selection, fee verification, and enrollment confirmation. The planning agent determines the complete sequence of actions required to achieve successful registration.

Specifications

Goal Formulation

Goal State:

Student successfully enrolled.

Performance Measure:

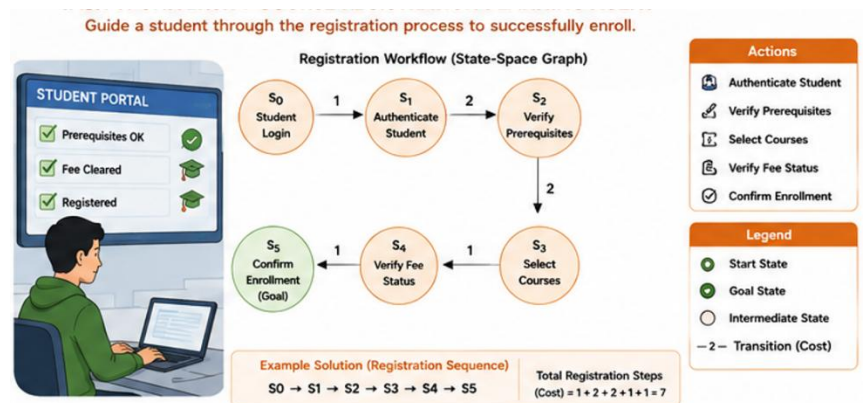
- Minimize registration steps.
- Ensure all academic requirements are satisfied.

Problem Formulation

Initial State: Student Login.

Possible Actions:

- Authenticate Student
- Verify Prerequisites
- Select Courses
- Verify Fee Status
- Confirm Enrollment



Transition Model:

Each completed task updates the registration state.

Goal Test:

Registration status equals **Completed**.

Path Cost:

Each registration step has a processing cost of one unit.

Implementation Requirements

Students shall:

- Represent the registration workflow as a graph.
- Generate all reachable states.
- Prevent revisiting completed registration steps.
- Display the execution sequence together with the total path cost.

UNGRADED HOME TASKS:

Task 4: Smart Airport Check-in Planning Agent

Objective

Design and implement a **Problem-Solving Agent** that assists passengers in completing the airport check-in process by planning a sequence of actions from arrival to boarding.

Scenario

Modern airports employ intelligent digital assistants to guide passengers through multiple stages of the departure process. Depending on the passenger's travel status, the system must determine an appropriate sequence of actions such as identity verification, baggage check-in, security screening, immigration (for international flights), and boarding. Since the environment is static during planning, the agent computes a complete plan before execution.

Specifications

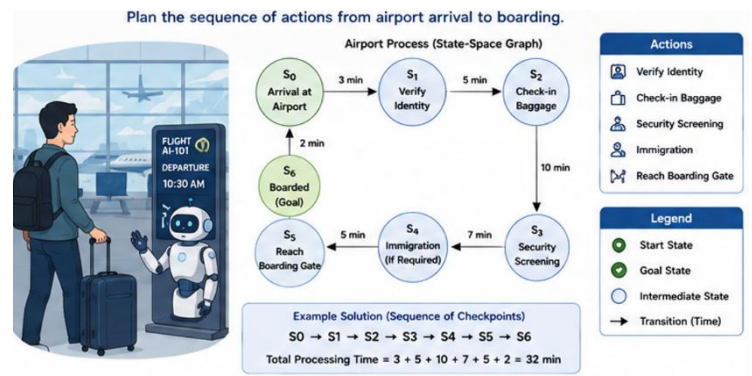
1. Goal Formulation

- Goal State: Passenger successfully boards the assigned flight.
- Performance Measure:
 - Minimize total processing time.
 - Avoid unnecessary checkpoints.

2. Problem Formulation

- **Initial State:** Passenger arrives at the airport entrance.
- **Possible Actions:**

- Verify Identity
- Check-in Baggage
- Pass Security Screening
- Complete Immigration
- Reach Boarding Gate



- **Transition Model:** Executing an action updates the passenger's current state.
- **Goal Test:** Passenger state equals **Boarded**.
- **Path Cost:** Each completed checkpoint has an associated processing time.

3. Implementation Requirements

Students shall:

- Represent airport checkpoints using an adjacency-list graph.
- Programmatically define:
 - Initial State
 - Actions
 - Transition Model
 - Goal Test
 - Path Cost
- Generate the reachable state space.
- Display the final sequence of checkpoints together with the total processing time.

4. Constraints

- Minimum 6 checkpoints.
- Static environment.
- Do not revisit completed checkpoints.
- Store visited states to avoid cycles.

Expected Output

Display:

- Initial State
- Goal State
- State-space graph
- Planned checkpoint sequence

Total processing time

Task 5: Smart Warehouse Inventory Retrieval Agent

Objective

Implement a **Problem-Solving Agent** that generates a sequence of warehouse movements required to retrieve an inventory item.

Scenario

A smart warehouse stores products across multiple storage zones connected through aisles. An autonomous retrieval system receives a product request and must determine the sequence of warehouse locations that should be visited before reaching the target storage rack. The planning process is performed offline using a known warehouse layout without real-time environmental changes.

Specifications

Goal Formulation

Retrieve the requested inventory item.

Performance Measure:

- Minimize travel distance.
- Avoid revisiting storage zones.

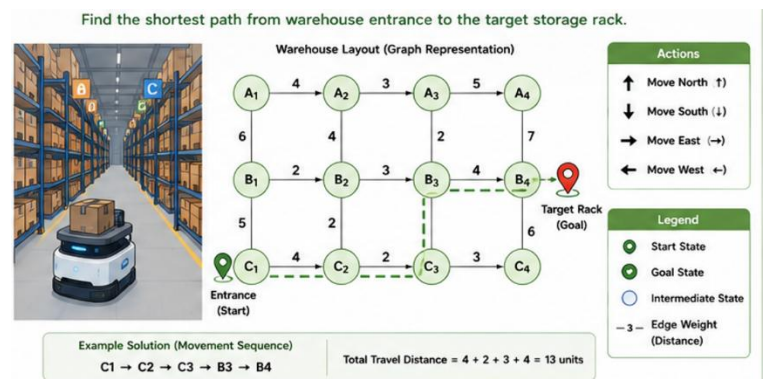
Problem Formulation

- Initial State: Warehouse Entrance
- Goal State: Target Storage Rack
- Possible Actions:
 - Move North
 - Move South
 - Move East
 - Move West

- Transition Model:
 - Movement changes the robot's current warehouse location.
- Goal Test:
 - Current location equals target rack.
- Path Cost:
 - Distance travelled between connected locations.

Implementation Requirements

Students shall:



- Represent warehouse locations using a graph.
- Generate reachable warehouse states.
- Prevent revisiting explored locations.
- Display the movement sequence and total travel cost.